

Major Project

OT Security Patch Management System

NodeMCU ESP32-S3-12K + NuttX

Documentation

Procured By:

Clarmen Yuen Zhi Kang (2105153G)

Quek Li Xuan Jacob (2104994E)

Isaac Tan (2100902B)

Hoi Shao Meng (2104655B)

Contents

Contents	2
Introduction	1
Pre-requisite	2
Initial Set up	3
Setting up with Ubuntu/Linux	4
Configuring Nuttx + MCUboot.....	5
Flashing NuttX into ESP32-S3	9
Configuring ESP32-S3 Access Point (AP).....	10
Configuring WLAN0.....	10
Configuring WLAN1.....	11
Connecting to the Access Point.....	11
Configuring a Blinking LED example	13
Over-The-Air (OTA) Updates Configuration (Steps).....	16
Initial Configurations.....	16
OTA using NuttX MCUBoot (Explanation)	23
MQTT Set Up + Security (Virtualized Network Environment on VirtualBox)	25
Prerequisites:	25
Steps for setting up Broker VM	27
Steps for setting up Subscriber ESP32	28
General Troubleshooting	30
Command not found.....	30
Port is busy or doesn't exist	31
References	32

Introduction

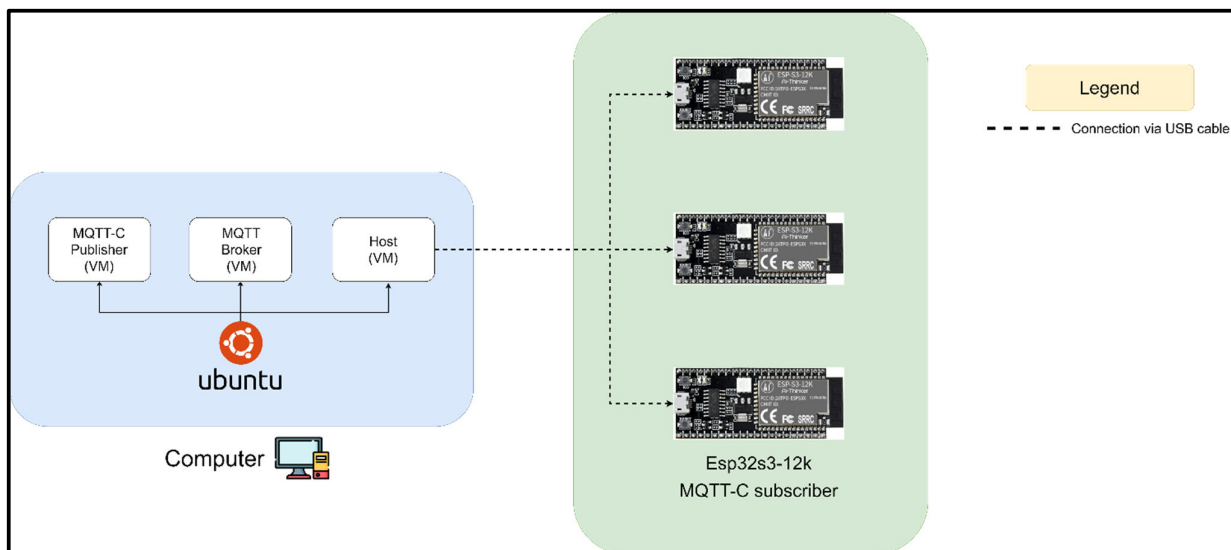
There have been several cases such as Solar Wind, Log4j, and Supplier Chain attack that has been happening in the past few years. It is especially risky today when we mention about invaded system in corporate settings, etc.

Objectives of the Project

Write a documentation and technical guide about utilising MCUboot, Nuttx, and ESP32-S3 to perform OTA (Over the Air) updates of the operating system and ensuring security measures such as encryption, hash values are verified against the original source before performing an operating system update on the target machine.

Scope of the Project

To make sure that whenever there is a new OS Patch/image, MCUboot compares the hash values of the received OS patch/image to a web server of the same OS Patch/image hash value and be verified before the whole thing is booted up to the new image.



Pre-requisite

Hardware Requirements:

- NodeMCU ESP32-S3-12K
- USB Type-B cable (Preferably 480MBPS Transfer speed and above)
- (OPTIONAL) USB to UART cable

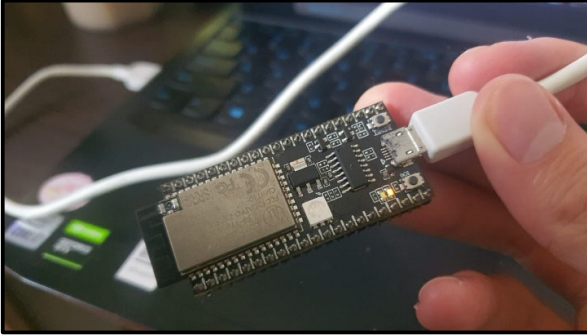
Software Requirements:

- Ubuntu/Kali Linux (or any other Debian-based Linux OS)
- Internet Connection
- (OPTIONAL) Mobile Hotspot

Initial Set up

USB Type-B Cable

Plug in the ESP32-S3 into your device with the USB Type B Cable through the USB port. Upon succession, your ESP32-S3 board should have a yellow LED lit up.



(Optional) USB to UART cable

If you do not have a Type B cable that can successfully connect into your device, you can use the UART cable as a substitute. In accordance, your UART cable must have the following pin types:

- TXD
- RXD
- GND
- +5V

Connect the wires from your UART to the ESP32-S3 accordingly:

UART Pin type	ESP32-S3 Pin
RXD	TX
TXD	RX
GND	GND
+5V	5V

Setting up with Ubuntu/Linux

Now that our device is plugged in, we need to ensure that our OS recognises it.

IF YOU ARE PLUGGING IN WITH A TYPE-B CABLE:

The device will be recognized as “QinHeng Electronics CH340 serial Converter”.

IF YOU ARE PLUGGING IN WITH A UART CABLE:

The device will be recognized as “Silicon Labs CP210x UART Bridge”.

```
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 006: ID 10c4:ea60 Silicon Labs CP210x UART Bridge
Bus 001 Device 005: ID 1a86:7523 QinHeng Electronics CH340 serial converter
```

Once the device is plugged in and recognized, it will be registered as “ttyUSB0”.

```
ty30  tty02  tty37  vmci-vsock
ty31  tty63  ttyS8  vmci
ty32  tty7   ttyS9  vsock
ty33  tty8   ttyUSB0 zero
ty34  tty9   udmabuf zfs
ty35  ttyprintk uhid
ty36  ttyS0  uinput
ty37  ttyS1  urandom
```

Configuring Nuttx + MCUboot

Firstly, we check and update any applications within the terminal.

```
sudo apt-get update
```

Then, install the required esptool application using pip.

```
sudo apt-get install python3-pip python-dev  
sudo pip install esptool
```

Once finished, we will then install the following tools required for further procedures.

```
sudo apt-get install git wget libncurses-dev flex bison gperf python3  
python3-pip python3-setuptools python3-serial python3-cryptography  
python3-future python3-pyparsing python3-pyelftools cmake ninja-build  
ccache libffi-dev libssl-dev dfu-util libusb-1.0-0 autoconf build-  
essential texinfo libtool libncurses5-dev gawk libc6-dev libtool-bin  
gettext libncursesw5-dev automake pkg-config libgmp-dev libmpc-dev  
libmpfr-dev libisl-dev binutils-dev libelf-dev libexpat-dev gcc-multilib  
g++-multilib picocom u-boot-tools util-linux -y
```

Once we finished installing all required tools, we will need to set up the tools.

```
mkdir ~/esp  
cd ~/esp  
git clone --recursive https://github.com/espressif/esp-idf.git  
cd ~/esp/esp-idf  
export IDF_GITHUB_ASSETS="dl.espressif.com/github_assets"  
./install.sh
```

Export the tools for terminal use (NOTE: this command is needed for every individual terminal).

```
. $HOME/esp/esp-idf/export.sh
```

Now that we have our configuring tools set up, we will now set up NuttX.

```
mkdir ~/nuttxspace  
cd ~/nuttxspace
```

NuttX uses KConfig for their configuration system, so we will need to install it.

If you are using Ubuntu 20.04 LTS or later:

```
sudo apt install kconfig-frontends
```

If you are using Ubuntu 18.04 or earlier, or kconfig is incompatible with your Linux OS, we will download the source and build that instead.

```
git clone https://bitbucket.org/nuttx/tools.git tools
cd tools/kconfig-frontends
./configure --enable-mconf --disable-nconf --disable-gconf --disable-qconf
make
make install
```

We will now install NuttX.

```
git clone https://github.com/apache/nuttx nuttx
git clone https://github.com/apache/nuttx-apps apps
```

We then need to initialise the hardware which allows us to tell NuttX what kind of configurations and settings we will need for our ESP32-S3 device. With the following commands, we can view all the available pre-built configurations for all supported devices under NuttX.

```
cd ~/nuttxspace/nuttx
./tools/configure.sh -L | less
```

As this documentation assumes you are using the NodeMCU ESP32-S3-12K, we will be using the following configuration:

```
./tools/configure.sh -l esp32s3-devkit:sta_softap
```

This pre-built configuration enables the Wi-Fi and Access Point (AP) functions within the board.

Now that we configured NuttX, we will need to enable MCUBoot and build NuttX.

```
make menuconfig
```

```
System Type > Application Image Configuration > MCUBoot-bootable format  
Application Configuration > Bootloader Utilities > MCUBoot
```

Save and close the menu.

Compile the configurations.

```
make clean
```

```
make
```

Now that we have a .bin for our board to be flashed into, we will need a bootloader binary file for our board. Since we are using MCUBoot, we will need its bootloader.

Go to <https://github.com/espressif/esp-nuttx-bootloader> and download the following file.

MCUboot bootloader	
Chip	Bootloader
ESP32	mcuboot-esp32.bin
ESP32-S2	mcuboot-esp32s2.bin
ESP32-S3	mcuboot-esp32s3.bin
ESP32-C3	mcuboot-esp32c3.bin

In the terminal, copy the downloaded `mcuboot-esp32s3.bin` file from `~/Downloads` into your NuttX working directory.

```
cp ~/Downloads/mcuboot-esp32s3.bin ~/nuttxspace/nuttx
```

Flashing NuttX into ESP32-S3

All that is left now is to Flash NuttX into our board, but there are some pre-requisites that we will need to configure before we do.

```
sudo adduser <your username> dialout
sudo usermod -aG dialout <your username>
sudo chmod a+rw /dev/ttyUSB0
```

This allows to have full control over our USB port communications.

Finally, it is time to flash it into our board.

```
make flash ESPTOOL_PORT=/dev/ttyUSB0 ESPTOOL_BINDIR=~/.nuttxspace/nuttx
```

Once done, we run the following command to enter our board.

```
sudo picocom -b 115200 /dev/ttyUSB0
```

Your terminal should look like the one below.

```
picocom v3.1

port is       : /dev/ttyUSB0
flowcontrol   : none
baudrate is   : 115200
parity is     : none
databits are  : 8
stopbits are  : 1
escape is     : C-a
local echo is : no
noinit is     : no
noreset is    : no
hangup is     : no
nolock is     : no
send_cmd is   : sz -vv
receive_cmd is : rz -vv -E
imap is       : 
omap is       : 
emap is       : crclrf,delbs,
logfile is    : none
initstring    : none
exit_after is : not set
exit is       : no

Type [C-a] [C-h] to see available commands
Terminal ready
*****j
                               Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x3fcd5a0,len:0x114c
load:0x403b2500,len:0x2b50
load:0x403ba000,len:0x157c
entry 0x403b2860
[esp32s3] [INF] Enabling RNG early entropy source ...
[esp32s3] [INF] ** Booting MCUboot build v1.9.0 **
[esp32s3] [INF] Primary image: magic=good, swap_type=0x1, copy_done=0x3, image_ok=0x1
[esp32s3] [INF] Scratch: magic=unset, swap_type=0x1, copy_done=0x3, image_ok=0x3
[esp32s3] [INF] Boot source: primary slot
[esp32s3] [INF] Swap type: none
[esp32s3] [INF] Disabling RNG early entropy source ...
[esp32s3] [INF] br_image_off = 0x10000
[esp32s3] [INF] ih_hdr_size = 0x20
[esp32s3] [INF] DRAM segment: start=0x2028f, size=0x16a0, vaddr=0x3fc9b390
[esp32s3] [INF] IRAM segment: start=0x183b7, size=0x7ed8, vaddr=0x40374000
[esp32s3] [INF] start=0x40374bf8

NuttShell (NSH) NuttX-12.4.0-RC0
nsh> |
```

Configuring ESP32-S3 Access Point (AP)

By now, you should be in the NuttX shell using the pre-built configuration `./tools/configure.sh -l esp32s3-devkit:sta_softap`.

From the `make menuconfig` configurations, we have registered a few commands into the Built-in Apps section of the NuttX shell. Enter `help` to view the available commands.

Enter `ifconfig` to view the network interfaces in the device. There should be 2 WLAN interfaces: WLAN0 and WLAN1.

WLAN0 is used to connect to a router, while WLAN1 is used to set up ESP32-S3 into an AP.

Note:

You must enable the bridged adapter in your VM settings to allow your VM/ESP32-S3 device to scan for local networks (your router/hotspot).

Configuring WLAN0

```
ifup wlan0
```

```
wapi psk wlan0 <passphrase> 3
```

```
wapi essid wlan0 <SSID> 1
```

```
renew wlan0
```

Where:

- `<passphrase>` and `<SSID>` are your router's information
- "3" and "1" are the wapi flags/modes (see `wapi help`)
- `renew wlan0` obtains a new IP address from the router via DHCP

To test:

- Ping from the NuttX shell to a google server (8.8.8.8)

```
nsh> ping -c3 8.8.8.8
PING 8.8.8.8 56 bytes of data
56 bytes from 8.8.8.8: icmp_seq=0 time=10.0 ms
56 bytes from 8.8.8.8: icmp_seq=1 time=10.0 ms
56 bytes from 8.8.8.8: icmp_seq=2 time=0.0 ms
3 packets transmitted, 3 received, 0% packet loss, time 3030 ms
rtt min/avg/max/mdev = 0.000/6.666/10.000/4.714 ms
```


Configuring WLAN1

As WLAN0 is being used to connect to the internet, WLAN1 will be used to set up the AP for the ESP32-S3 end devices to connect, to receive files through MQTT.

```
ifup wlan1
```

```
dhcpd_start wlan1
```

```
wapi psk wlan1 mypasswd 3
```

```
wapi essid wlan1 nuttxap 1
```

Where:

- `dhcpd_start` associates an IP address to a connected device
- “myswrd” is your new AP <passphrase> and “nuttxap” is your new AP <ESSID>

Connecting to the Access Point

Create another VM for your ESP32-S3 end device and follow the same steps above in getting the NuttX shell. However, use the pre-built configuration `./tools/configure.sh -l esp32s3-devkit:wifi` this time. This pre-built configuration only enables the Wi-Fi station in the board, without the AP feature.

Once the AP is set up, a network SSID called “nuttxap” (or whatever you named it as) will appear. Connect to it on your second VM by entering the same credentials you configured in the NuttX shell previously.

```
ifup wlan0
```

```
wapi psk wlan0 mypasswd 3
```

```
wapi essid wlan0 nuttxap 1
```

```
renew wlan0
```

You will also be automatically assigned an IP address of 10.0.0.X.

```
nsh> ifup wlan0
ifup wlan0 ... OK
nsh> wapi psk wlan0 mypasswd 3
nsh> wapi essid wlan0 nuttxap 1
nsh> renew wlan0
nsh> ifconfig
wlan0    Link encap:Ethernet HWaddr f4:12:fa:43:60:e0 at RUNNING mtu 1500
         inet addr:10.0.0.3 Bcast:10.0.0.1 Mask:255.255.255.0
```

To test:

- Ping to 10.0.0.1

```
nsh> ping -c3 10.0.0.1
PING 10.0.0.1 56 bytes of data
56 bytes from 10.0.0.1: icmp_seq=0 time=10.0 ms
56 bytes from 10.0.0.1: icmp_seq=1 time=0.0 ms
56 bytes from 10.0.0.1: icmp_seq=2 time=10.0 ms
3 packets transmitted, 3 received, 0% packet loss, time 3030 ms
rtt min/avg/max/mdev = 0.000/6.00/4.714 ms
```

Unfortunately, though the AP is set up and running, there is no internet going through for the end device. As seen below, it is unable to ping to a google server.

```
nsh> ping -c3 8.8.8.8
PING 8.8.8.8 56 bytes of data
No response from 8.8.8.8: icmp_seq=0 time=1000 ms
No response from 8.8.8.8: icmp_seq=1 time=1000 ms
No response from 8.8.8.8: icmp_seq=2 time=1000 ms
3 packets transmitted, 0 received, 100% packet loss, time 3030 ms
```

As we suspect that this may be another `menuconfig` dependency issue, this is the furthest in setting up the board into an AP in attempt to fulfil the project requirements. In the following MQTT setup, we will not be using the AP. Instead, the end device will be connected to a local router or mobile hotspot using the same steps to perform the file transfer.

Configuring a Blinking LED example

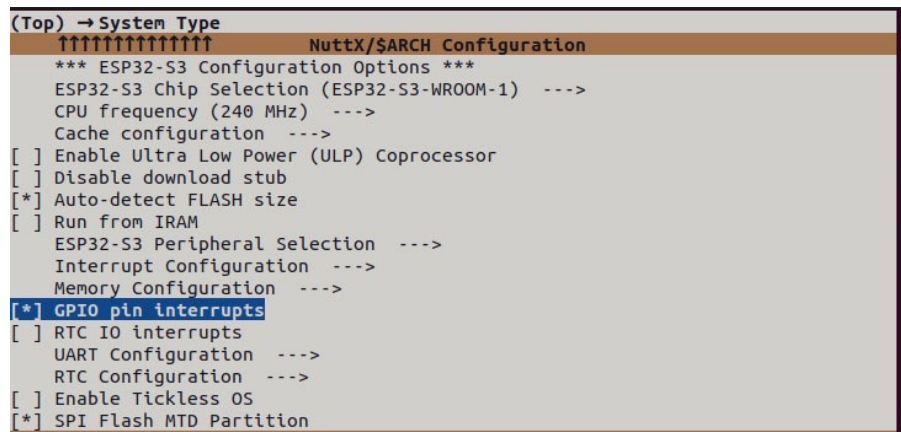
Now, to test if we can successfully implement new features through OTA patching, we will be writing a LED script that allows the board to blink.

As NodeMCU ESP32-S3-12K does not support the current NuttX LED Driver, we will be using the GPIO driver instead.

Firstly, head back to the NuttX directory and enable GPIO.

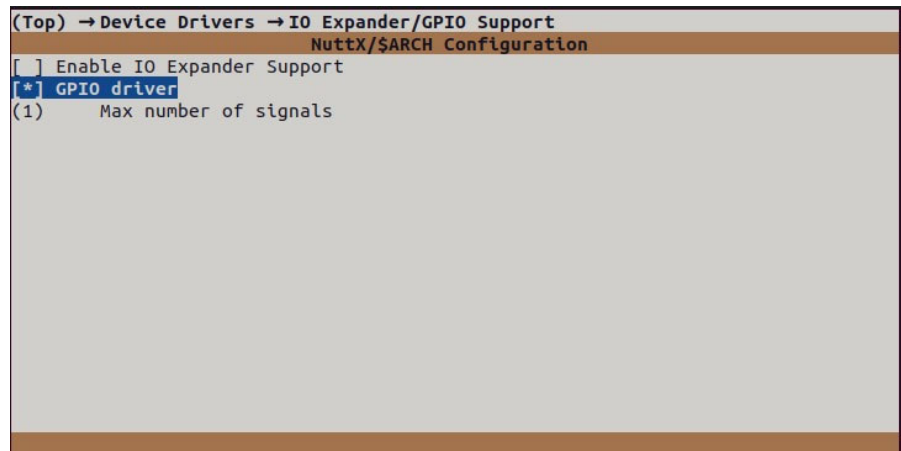
```
make menuconfig
```

```
System type > GPIO Pin interrupts
```



```
(Top) → System Type
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑ NuttX/$ARCH Configuration
*** ESP32-S3 Configuration Options ***
ESP32-S3 Chip Selection (ESP32-S3-WROOM-1) --->
CPU frequency (240 MHz) --->
Cache configuration --->
[ ] Enable Ultra Low Power (ULP) Coprocessor
[ ] Disable download stub
[*] Auto-detect FLASH size
[ ] Run from IRAM
ESP32-S3 Peripheral Selection --->
Interrupt Configuration --->
Memory Configuration --->
[*] GPIO pin interrupts
[ ] RTC IO interrupts
[ ] UART Configuration --->
[ ] RTC Configuration --->
[ ] Enable Tickless OS
[*] SPI Flash MTD Partition
```

```
Device Drivers > IO Expander/GPIO Support > GPIO Driver
```



```
(Top) → Device Drivers → IO Expander/GPIO Support
NuttX/$ARCH Configuration
[ ] Enable IO Expander Support
[*] GPIO driver
(1) Max number of signals
```

Application Configurations > Examples > GPIO Driver example

```
(Top) → Application Configuration → Examples
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑
Nuttx/$ARCH Configuration
[ ] CORDIC example
[ ] DHCP server example
[ ] Dhtxx sensor example
[ ] ESP32 HIMEM Example
[ ] FM Synthesizer examples
[ ] FTP client example
[ ] FTP server example
[ ] FXOS8700CQ motion sensor example
[*] GPIO driver example
(gpio) Program name
(100) GPIO task priority
(2048) GPIO stack size
[ ] GPS example
[ ] HDC1008 driver example
[ ] "Hello, World!" example
[ ] "Hello, World!" example (Nim)
[ ] "Hello, Rust!" example
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑
```

Save and close the menu.

We will then need to specify which pin type we want the script to activate.

```
cd ~/nuttxspace/nuttx/boards/xtensa/esp32s3/esp32s3-devkit/src
vim esp32s3_gpio.c
```

Firstly, we will be using the boards Red RGB light as our blinker. The pin type that this particular LED is using is PIN TYPE 5, and we will need to define it as below.

```
#include <nuttx/config.h>
#include <sys/types.h>
#include <nuttx/lrq.h>
#include <arch/lrq.h>
#include <assert.h>
#include <debug.h>

#include <nuttx/loexpander/gpio.h>
#include <arch/board/board.h>

#include "esp32s3-devkit.h"
#include "esp32s3_gpio.h"
#include "hardware/esp32s3_gpio_sigmap.h"

#if defined(CONFIG_DEV_GPIO) && defined(CONFIG_GPIO_LOWER_HALF)
/* =====
 * Pre-processor Definitions
 * ===== */

/* Output pins. GPIO15 is used as an example, any other outputs could be
 * used.
 */
#define GPIO_OUT1 5

/* Input pins. GPIO18 is used as an example, any other inputs could be
 * used.
 */
#define GPIO_IN1 0

/* Interrupt pins. GPIO21 is used as an example, any other inputs could be
 * used.
 */
#define GPIO_IRQPIN1 21
```

We also need to add a custom script

```
#if BOARD_NGPIPIN > 0
for (i = 0; i < BOARD_NGPIPIN; i++)
{
    /* Setup and register the GPIO pin */

    g_gpin[i].gpio.gp_pintype = GPIO_INPUT_PIN;
    g_gpin[i].gpio.gp_ops     = &gpin_ops;
    g_gpin[i].id              = i;
    gpio_pin_register(&g_gpin[i].gpio, pincount);

    /* Configure the pins that will be used as INPUT */
    esp32s3_configgpio(g_gpioinputs[i], INPUT_FUNCTION_2);

    while(1){
        esp32s3_gpiowrite(g_gpiooutputs[i], 1);
        usleep(700000);
        esp32s3_gpiowrite(g_gpiooutputs[i], 0);
        usleep(700000);
        esp32s3_gpiowrite(g_gpiooutputs[i], 1);
    }

    pincount++;
}
#endif

#if BOARD_NGPIPOINT > 0
for (i = 0; i < BOARD_NGPIPOINT; i++)
{
```

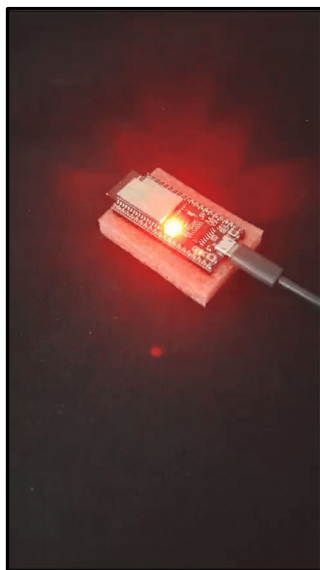
Once we added our script, we can recompile our NuttX folder and flash the file once again.

```
make clean
```

```
make
```

```
make flash ESPTOOL_PORT=/dev/ttyUSB0 ESPTOOL_BINDIR=~/.nuttxspace/nuttx
```

The board will now constantly blink the red LED



Over-The-Air (OTA) Updates Configuration (Steps)

Unfortunately, due to circumstances beyond our control, we are unable to implement this feature within the given time frame for this project.

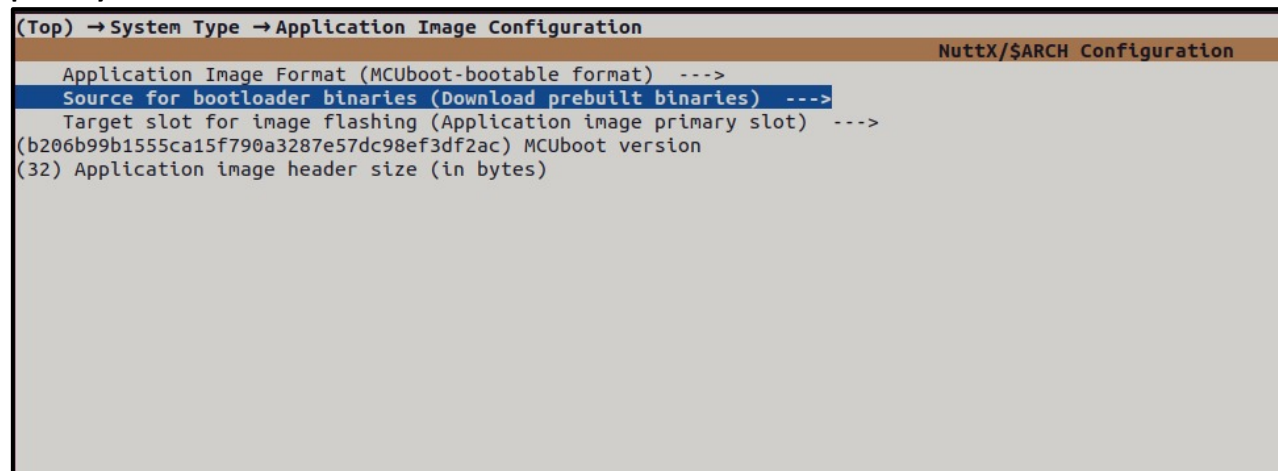
Due to the lack of support and online documentation regarding this feature for the ESP32-S3, it is difficult to pin-point the exact problem as to why this feature cannot be implemented within the device. After thorough investigations, we have deemed the issue to be one of two possible events.

Initial Configurations

Enter `make menuconfig` within the NuttX directory

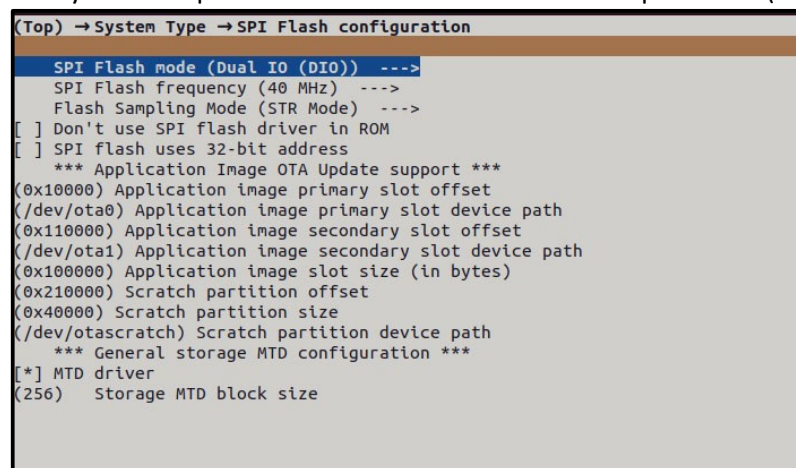
System Type > Application Image Configuration

Ensure that Image format is “MCUboot-bootable format” and target slot is “Application image primary slot”



Then head to System Type > SPI Flash Configuration

Verify the OTA partitions ROM address and Scratch partitions (default values)



Application Configuration > Bootloader Utilities

Verify that MCUboot is enabled and that firmware image slot path are as specified.

```
(Top) → Application Configuration → Bootloader Utilities → MCUboot
Nuttx/$ARCH Configuration
(fefc398cc13ebbc527e297fe9df78cd98a359d75) MCUboot version
[ ] Enable MCUboot logging
    Cryptographic backend (TinyCrypt) --->
[ ] MCUboot bootloader application
    Upgrade mode (A/B image swapping with rollback) --->
    Swap upgrade policy (Image swap using scratch area) --->
(/dev/ota0) Application firmware image primary slot path
(/dev/ota1) Application firmware image secondary slot path
(/dev/otascratch) Scratch partition path
(0xff) Default flash erase state
```

Application Configuration > Examples > MCUboot Examples

Verify that MCUboot update agent example is enabled.

```
(Top) → Application Configuration → Examples → MCUboot Examples
Nuttx/$ARCH Configuration
[ ] MCUboot slot confirm example
[ ] MCUboot SWAP Test image example
[*] MCUboot update agent example
() URL for update image
(512) Download buffer size in bytes
```

Application Configuration > NSH Library > NSH Library > Message of the day (MOTD)

For analogy purposes, we will be giving the shell a MOTD message.

```
(Top) → Application Configuration → NSH Library → NSH Library → Message of the Day (MOTD)
Nuttx/$ARCH Configuration
[ ] Platform MOTD (NEW)
(Welcome to Nuttx from MCUboot) MOTD String
```


Application Configuration > Network Utilities

Enable the uIP web client.

```
(Top) -> Application Configuration -> Network Utilities
|||||
[*] Iperf example
(Iperf) Program name
(100) Iperf task priority
(4096) Iperf stack size
(wlan0) Iperf Network device
[ ] Iptables library
[ ] JSONCPP
[ ] CURL4nx HTTP client library
[*] Enable MQTT-C
[*] Enable MQTT-C example
(8192) Task's stack size
[ ] Enable MQTT-C with mbedtls
(1.1.5) MQTT-C Version
[ ] NetCat tool
*- Network initialization --->
*- Network support library
*- Build the generic URL parser
[ ] Enable Nanomsg NG
[ ] NTP client
*- ICMP ping support
[ ] PPP server
[ ] Remote Execution Server
[ ] Remote execution client
[ ] Enable RTP Tools
[ ] SMTP
[ ] Telnet client library
[ ] Telnet daemon
[ ] TFTP client
[ ] WakeOnLAN support
[*] uIP web client
(NuttX/6.xx.x (; http://www.nuttX.org/)) wget User Agent
(10) Request and receive timeouts
(200) Max http line size
(32) Max line size
(40) Max host name size
(100) Max file name size
[ ] uIP web server
[ ] XML RPC library
```

Compile and flash the new nuttx.bin file into the ESP32-S3, upon finishing, the final product should look like the following.

```
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x3fcd5a0,len:0x114c
load:0x403b2500,len:0x2b50
load:0x403ba000,len:0x157c
entry 0x403b2860
[esp32s3] [INF] Enabling RNG early entropy source...
[esp32s3] [INF] *** Booting MCUboot build v1.9.0 ***
[esp32s3] [INF] Primary image: magic=good, swap_type=0x1, copy_done=0x3, image_o
k=0x1
[esp32s3] [INF] Scratch: magic=unset, swap_type=0x1, copy_done=0x3, image_ok=0x3
[esp32s3] [INF] Boot source: primary slot
[esp32s3] [INF] Swap type: none
[esp32s3] [INF] Disabling RNG early entropy source...
[esp32s3] [INF] br_image_off = 0x10000
[esp32s3] [INF] ih_hdr_size = 0x20
[esp32s3] [INF] DRAM segment: start=0x1e26c, size=0x2094, vaddr=0x3fc99670
[esp32s3] [INF] IRAM segment: start=0x16da0, size=0x74cc, vaddr=0x40374000
[esp32s3] [INF] start=0x40374b78

NuttShell (NSH) NuttX-12.3.0
Welcome to NuttX from MCUboot
nsh>
```


Showing we have `mcuboot_agent` flashed in.

```
nsh> help
help usage:  help [-v] [<cmd>]

.      cp      false   mkdir     rm        umount
[      cmp     fdinfo   mkrd      rmdir     unset
?      dirname free     mount     set       uptime
alias  dd       help     mv        sleep     usleep
unalias df       hexdump  nslookup source    wget
arp     dmesg   ifconfig pidof     test      xd
basename echo     ifdown   printf    time
break   env     ifup     ps        true
cat     exec    kill     pwd       truncate
cd      exit    ls       reboot    uname

Builtin Apps:
dhcpd      iperf      ping      sh
dhcpd_start mcuboot_agent rand      wapi
dhcpd_stop  nsh       renew
```

Before we proceed, please know that the following steps are procreated test steps to see if the OTA update works.

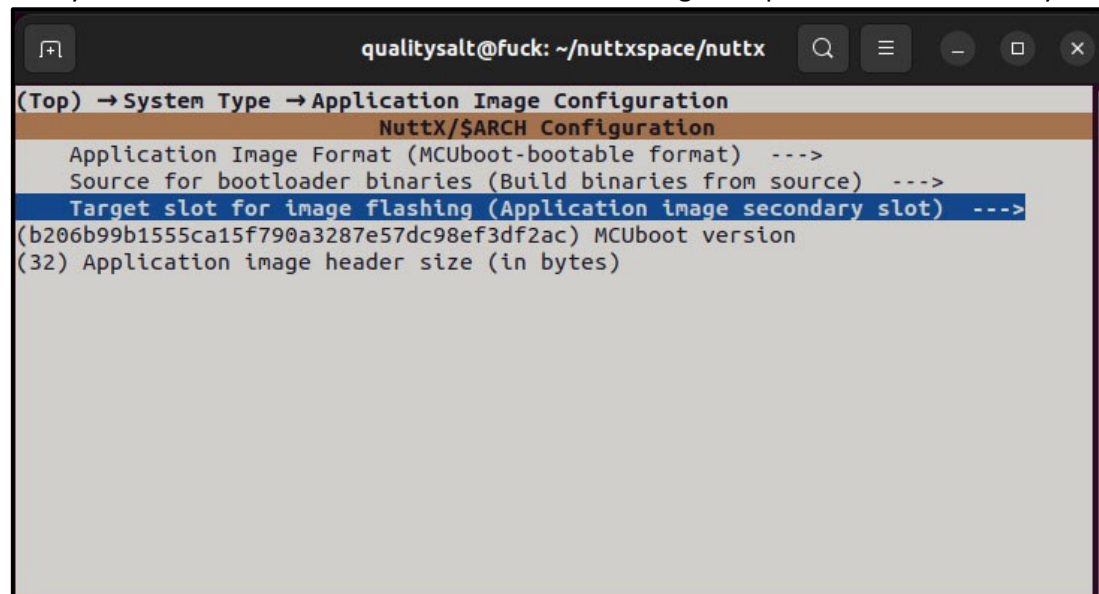
`make distclean`

`./tools/configure.sh -l esp32s3-devkit:mcuboot_nsh`

`make menuconfig`

Application Configuration > Bootloader Utilities

Verify that MCUBoot is enabled and that firmware image slot path is set to secondary slot.



Application Configuration > Bootloader Utilities

Verify that MCUboot is enabled and that firmware image slot path are as specified.

```
(Top) → Application Configuration → Bootloader Utilities → MCUboot
Nuttx/$ARCH Configuration
(fefc398cc13ebbc527e297fe9df78cd98a359d75) MCUboot version
[ ] Enable MCUboot logging
    Cryptographic backend (TinyCrypt) --->
[ ] MCUboot bootloader application
    Upgrade mode (A/B image swapping with rollback) --->
    Swap upgrade policy (Image swap using scratch area) --->
(/dev/ota0) Application firmware image primary slot path
(/dev/ota1) Application firmware image secondary slot path
(/dev/otascratch) Scratch partition path
(0xff) Default flash erase state
```

Application Configuration > Examples > MCUboot Examples

Verify that MCUboot slot confirm example is enabled.

```
(Top) → Application Configuration → Examples → MCUboot Examples
Nuttx/$ARCH Configuration
[*] MCUboot slot confirm example
[ ] MCUboot SWAP Test image example
```

Save and close the menu.

```
make clean
```

```
make
```

Once the file has been compiled, DO NOT FLASH IT INTO THE ESP32S3.

The newly compiled nuttx.bin will serve as the new .bin file we flash into our device using OTA, just for reference, we will copy and rename the file.

```
cp nuttx.bin ota_update.bin
```

As we can see, the signature of the file is still same to ensure validity.

```
~/nuttxspace/nuttx$ md5sum nuttx.bin
f7fdaaf16e3a81e43e3aa59c46a7485e  nuttx.bin
~/nuttxspace/nuttx$ md5sum ota_update.bin
f7fdaaf16e3a81e43e3aa59c46a7485e  ota_update.bin
~/nuttxspace/nuttx$
```

We then host a web server for the esp32s3 to receive its update

```
sudo python3 -m http.server 8080
```

```
~/nuttxspace/nuttx$ sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Entering our local host address in the web browser will show us that our update file is present.



With all pre-requisites finished, we can now enter our ESP32-S3 and attempt an OTA update.

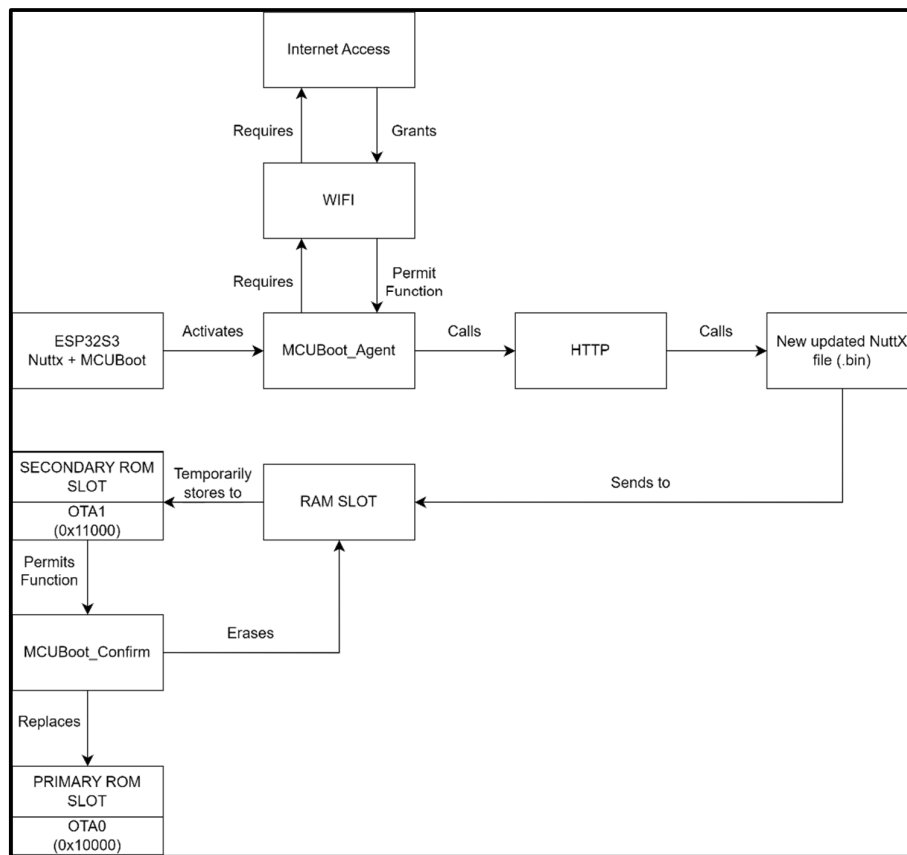
However, while attempting to do so, our group received an error under the 'MCUBoot_Agent' application, a message saying "Failed to open Flash area" before terminating the process altogether.

```
nsh> mcuboot_agent http://192.168.1.100/ota_update.bin
MCUboot Update Agent example
Downloading from http://192.168.1.100/ota_update.bin
Failed to open flash area
Application Image download failure.
Update aborted.
nsh> █
```

Despite numerous attempts with consideration to our limited time frame, we failed to find a fix to this error.

OTA using NuttX MCUBoot (Explanation)

Below shows our group's current understanding of how OTA works using NuttX:



NuttX provides a working example of the OTA functionalities with MCUboot verification and to our understanding, this is how the process works. Despite our best efforts, we had received an error where no other had encountered, and we simply could not resolve this issue within the time frame we are given.

Initially, we had assumed that we had allocated the wrong ROM address for the OTA procedures and had changed the address within the defined space under '/dev/ota0' and '/dev/ota1'. However, further testing has proven that the default flashing area memory space is defined under '/dev/ota0', which means that the defined ROM address is correct.

Further investigations lead us to assume that there is instead an undefined flash partition for the OTA to call its ROM space for its function work, which suggest a SPI Flash error. But despite our best efforts, we are unable to find any further leads which defines which partition is causing the error.

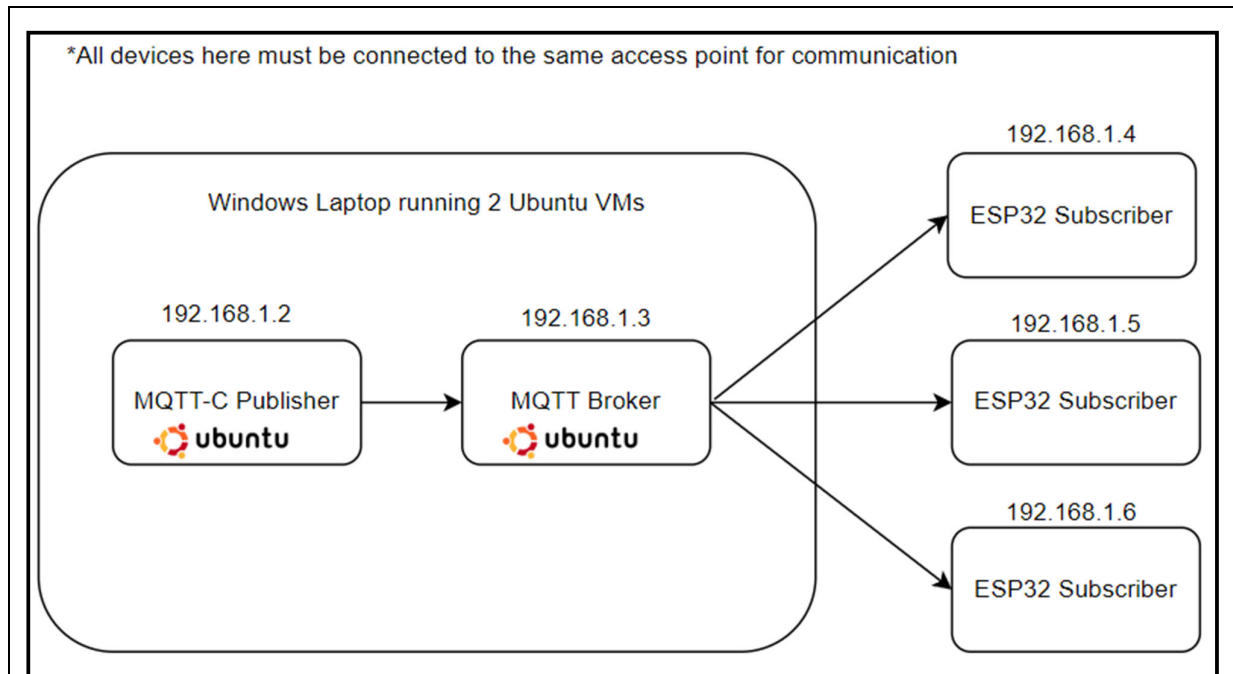
Reading the applications source code shows us 2 variables that needs to be declared beforehand.

```
ret = flash_area_open(FLASH_AREA_IMAGE_SECONDARY(0), &dl_ctx.fa);  
if (ret != OK)  
{  
    fprintf(stderr, "Failed to open flash area\n");  
  
    return ret;  
}
```

```
dl_ctx.fa = NULL;  
dl_ctx.fa_offset = 0;  
dl_ctx.image_size = -1;
```

However, we were unable to pinpoint any errors related to these 2 variables.

MQTT Set Up + Security (Virtualized Network Environment on VirtualBox)



Prerequisites:

- 2 VMs running Ubuntu – Subscriber and Broker (if you are using broker on your host VM).
- All VMs connected in the same local network.

Steps for setting up Publisher VM

Install all necessary building tools and libraries using the package manager.

```
sudo apt install build-essential git cmake
```

Navigate to <https://nuttx.apache.org/download/> and install the latest OS/Apps Version.

Save the NuttX.bin file and remember where its stored.

Clone the MQTT-C repository from GitHub.

```
git clone https://github.com/LiamBindle/MQTT-C.git
```

Navigate into the cloned directory to prepare the build environment and compile it.

```
cd MQTT-C
```

```
mkdir build
```

```
cd build
```

```
cmake ..
```

```
make
```

Modify the provided example publisher code to point to your MQTT broker's IP address and set the desired topic.

```
nano ~/MQTT-C/examples/simple_publisher.c
```

In our case, it is `54.210.203.95` and `nuttx/updates`.

```
/* get address (argv[1] if present) */
if (argc > 1) {
    addr = argv[1];
} else {
    addr = "54.210.203.95";
}

/* get port number (argv[2] if present) */
if (argc > 2) {
    port = argv[2];
} else {
    port = "1883";
}

/* get the topic name to publish */
if (argc > 3) {
    topic = argv[3];
} else {
    topic = "nuttx/updates";
}
```


Add appropriate code to simple_publisher.c to publish .bin files when executed.

```
while(fgetc(stdin) == '\n') {
// Open the binary file
const char* filename = "/home/shao/Downloads/mcuboot-esp32s3.bin"; // Replace with the path to your .bin file
FILE* file = fopen(filename, "rb");
if (file == NULL) {
    perror("Failed to open file");
    exit_example(EXIT_FAILURE, sockfd, &client_daemon);
}
```

Create an executable file of simple_publisher.c that we just modified.

```
cd ~/MQTT-C/examples/
```

```
gcc -o simple_publisher simple_publisher.c -I../include -L../build -lmqttc -pthread
```

Publish the .bin file.

```
./simple_publisher
```

Steps for setting up Broker VM

Install all necessary mosquitto tool

```
sudo apt-get update
```

```
sudo apt-get install mosquitto
```

Install all necessary building tools and libraries using the package manager

(If unable to access folder with this line, go to each directory one by one)

```
nano ~/etc/mosquitto/mosquitto.conf
```

Add the following lines:

```
listener 1883      #Listen on port 1883
allow_anonymous true #Allow connections from external sources to connect to broker
```

Restart the mosquitto service to apply the changes.

```
sudo systemctl restart mosquitto
```

Ensure mosquitto service is running.

```
sudo systemctl status mosquitto
```

Steps for setting up Subscriber ESP32

In your NuttX working directory, choose the following configuration command, this enables the Wi-Fi feature in the device to connect to your local router.

```
./tools/configure.sh -l esp32s3-devkit:wifi
```

Enable the following in `make menuconfig`:

System Type > Bootloader and Image Configuration > MCUboot-bootable
format

Library Routines > Have C++ compiler > Have C++ initialization

Application Configuration > Bootloader Utilities > MCUboot

Application Configuration > Network Utilities > Enable MQTT-C > Enable
MQTT-C example

Before flashing, configure the `simple_subscriber.c` file with the appropriate parameters to receive .bin files.

```
vim ~/nuttxspace/apps/netutils/mqttd/MQTT-C/examples/simple_subscriber.c
```

```
123 void publish_callback(void** unused, struct mqtt_response_publish *published) {
124     /* note that published->topic_name is NOT null-terminated (here we'll change it to a c-string) */
125     char* topic_name = (char*) malloc(published->topic_name_size + 1);
126     memcpy(topic_name, published->topic_name, published->topic_name_size);
127     topic_name[published->topic_name_size] = '\0';
128
129     /* Save the received binary data to a file */
130     const char* filename = "mcuboot-esp32s3.bin"; // Specify the filename to save the binary data
131     FILE* file = fopen(filename, "wb"); // Open the file in binary write mode
132
133     if (file == NULL) {
134         perror("Failed to open file");
135         free(topic_name);
136         return;
137     }
138
139     fwrite(published->application_message, 1, published->application_message_size, file);
140     fclose(file);
141
142     printf("Received publish('%s') and saved to '%s'\n", topic_name, filename);
143     free(topic_name);
144 }
145 }
```

Flash it into ESP32-S3.

```
make flash ESPTOOL_PORT=/dev/ttyUSB0 ESPTOOL_BINDIR=~/.nuttxspace/nuttx
```

In your NuttX shell, configure WLAN0 for internet connectivity, detailed explanation as documented above.

```
ifup wlan0
```

```
wapi psk wlan0 <passphrase> 3
```

```
wapi essid wlan0 <SSID> 1
```

```
renew wlan0
```

Test internet connectivity:

```
ping 8.8.8.8
```

Running `mqttc_posix_sub` will run the `simple_publisher.c` file in ESP32-S3.

```
mqttc_posix_sub
```

```
nsh> mqttc_posix_sub
mqttc_posix_sub listening for 'nuttx/updates' messages.
Press CTRL-D to exit.
```

Now, ESP32-S3 is ready and listening for updates.

General Troubleshooting

Command not found

Command ran:

```
make flash ESPTOOL_PORT=/dev/ttyUSB0 ESPTOOL_BINDIR=~/.NuttX/nuttx
```

```
admin-usr@adminusr-virtual-machine:~/NuttX/nuttx$ make flash ESPTOOL_PORT=/dev/t
tyUSB0 ESPTOOL_BINDIR=~/.NuttX/nuttx
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
```

Error message:

```
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
/usr/bin/bash: line 1: xtensa-esp32s3-elf-gcc: command not found
CPP: /home/admin-usr/NuttX/nuttx/boards/xtensa/esp32s3/esp32s3-devkit/./common
/scripts/esp32s3_peripherals.ld-> /home/admin-usr/NuttX/nuttx/boards/xtensa/esp3
2s3/esp32s3-devkit/./common/scripts/esp32s3_peripherals.ld.tmp /usr/bin/bash: l
ine 1: xtensa-esp32s3-elf-gcc: command not found
make[1]: *** [Makefile:169: /home/admin-usr/NuttX/nuttx/boards/xtensa/esp32s3/es
p32s3-devkit/./common/scripts/esp32s3_peripherals.ld.tmp] Error 127
make: *** [tools/Unix.mk:545: nuttx] Error 2
```

Solution:

```
. $HOME/esp/esp-idf/export.sh
```

```
admin-usr@adminusr-virtual-machine:~/NuttX/nuttx$ . $HOME/esp/esp-idf/export.sh
Setting IDF_PATH to '/home/admin-usr/esp/esp-idf'
Detecting the Python interpreter
Checking "python3" ...
Python 3.10.12
"python3" has been detected
Checking Python compatibility
Checking other ESP-IDF versions
```

Port is busy or doesn't exist

Command ran:

```
make flash ESPTOOL_PORT=/dev/ttyUSB0 ESPTOOL_BINDIR=~/.Nuttx/nuttx
```

```
no configuration change.  
admin-usr@adminusr-virtual-machine:~/Nuttx/nuttx$ make flash ESPTOOL_PORT=/dev/t  
tyUSB0 ESPTOOL_BINDIR=~/.Nuttx/nuttx
```

Error message:

```
A fatal error occurred: Could not open /dev/ttyUSB0, the port is busy or doesn't  
exist.  
([Errno 13] could not open port /dev/ttyUSB0: [Errno 13] Permission denied: '/de  
v/ttyUSB0')  
  
Hint: Try to add user into dialout group: sudo usermod -a -G dialout $USER  
  
make: *** [tools/Unix.mk:585: flash] Error 2
```

Solution:

<https://stackoverflow.com/questions/73923341/unable-to-flash-esp32-the-port-doesnt-exist>

add yourself to dialout and change permissions on it

```
$ sudo adduser <username> dialout  
$ sudo chmod a+rw /dev/ttyUSB0
```

```
admin-usr@adminusr-virtual-machine:~/Nuttx/nuttx$ sudo adduser admin-usr dialout  
Adding user 'admin-usr' to group 'dialout' ...  
Adding user admin-usr to group dialout  
Done.  
admin-usr@adminusr-virtual-machine:~/Nuttx/nuttx$ sudo chmod a+rw /dev/ttyUSB0
```

References

EmbeTronicX. (2022, June 29). *Setup NuttX RTOS with ESP32 Dev Board*.

<https://embetronicx.com/tutorials/rtos/nuttX/setup-nuttX-rtos-on-esp32-dev-board/>

EmbeTronicX. (2022, June 29). *ESP32 GPIO example using NuttX RTOS (GPIO Interrupt)*.

<https://embetronicx.com/tutorials/rtos/nuttX/esp32-gpio-example-using-nuttX-rtos-gpio-interrupt/>

Espressif ESP32-S3. (n.d.). *NuttX latest documentation*.

<https://nuttx.apache.org/docs/latest/platforms/xtensa/esp32s3/index.html>

Espressif Systems. (n.d.). *ESP-IDF Programming Guide*.

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/get-started/index.html>

<https://github.com/lupyuen/lupyuen.github.io/blob/master/src/nuttX.md>